

Experiment 5: Design and Implementation of Pipelined FIR Filter

Aslin Garvasis
EE24BTECH11008

1 Objective

To design and implement a pipelined FIR filter using Verilog, analyze hardware resource utilization using Quartus, and verify the output using MATLAB.

2 FIR Filter Theory

The FIR filter output is given by:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n-k]$$

3 Design Methodology

The experiment was carried out in the following steps:

1. FIR filter was designed using Verilog for both pipelined and non-pipelined architectures.
2. The designs were synthesized in Quartus Prime Lite to obtain resource utilization.
3. Testbenches were created to apply input signals and generate output data in text files.
4. The output data from Verilog simulation was processed using MATLAB.
5. MATLAB was used to plot the output signals for analysis.

4 Pipelined FIR Architectures

Three pipelined FIR architectures were implemented:

- Direct Form Pipeline
- Optimized Form Pipeline
- Genvar-Based Pipeline

Pipeline registers were inserted between stages to improve performance and reduce critical path delay.

5 Simulation and Output Generation

Testbenches were used to provide input signals (950 Hz, 1100 Hz, and 2000 Hz) to the FIR filter.

The simulation was carried out using ModelSim, and the output values were written into text files.

6 MATLAB Plotting

The output text files generated from Verilog simulation were imported into MATLAB.

The data was converted from fixed-point format to floating-point representation, and output waveforms were plotted.

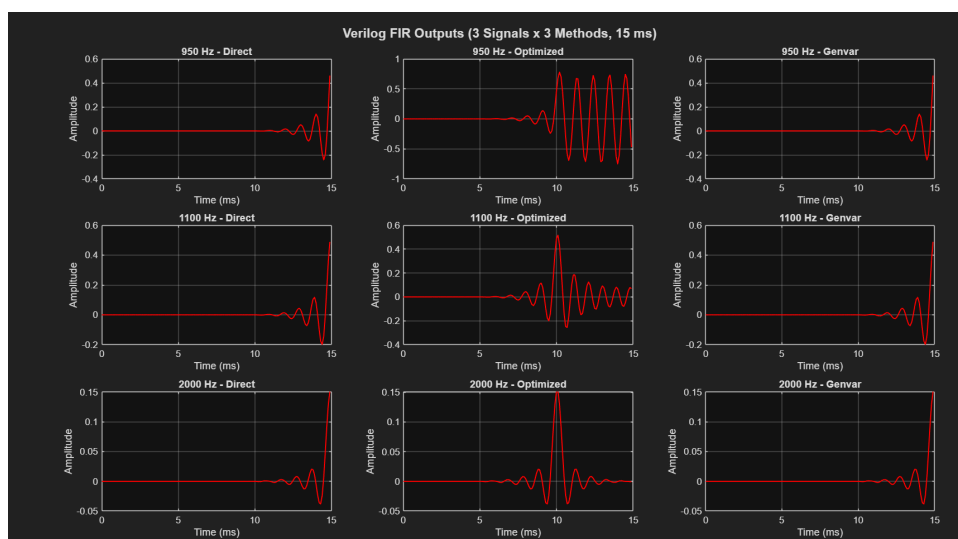
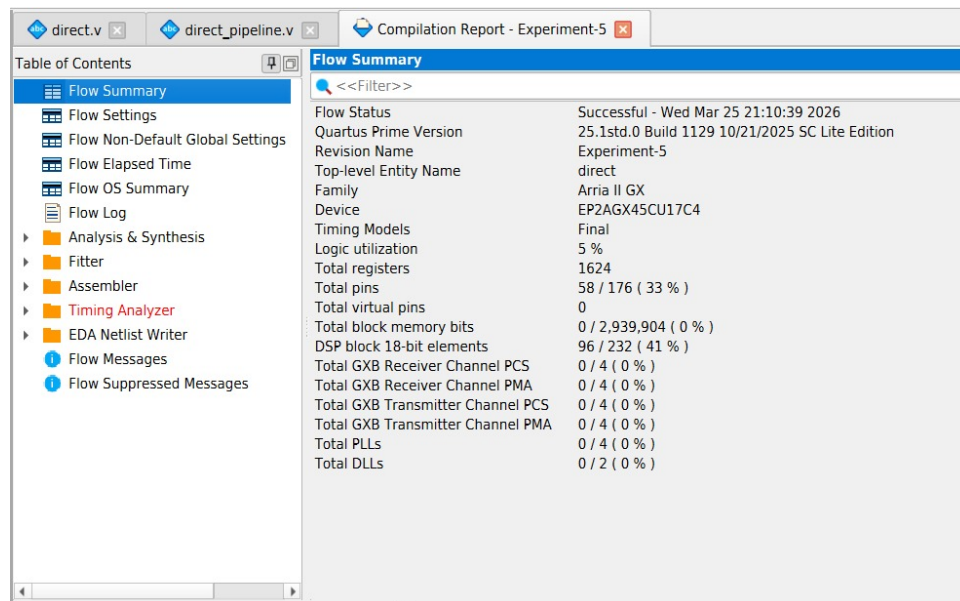


Figure 1: Output waveform obtained from MATLAB

7 Resource Utilization

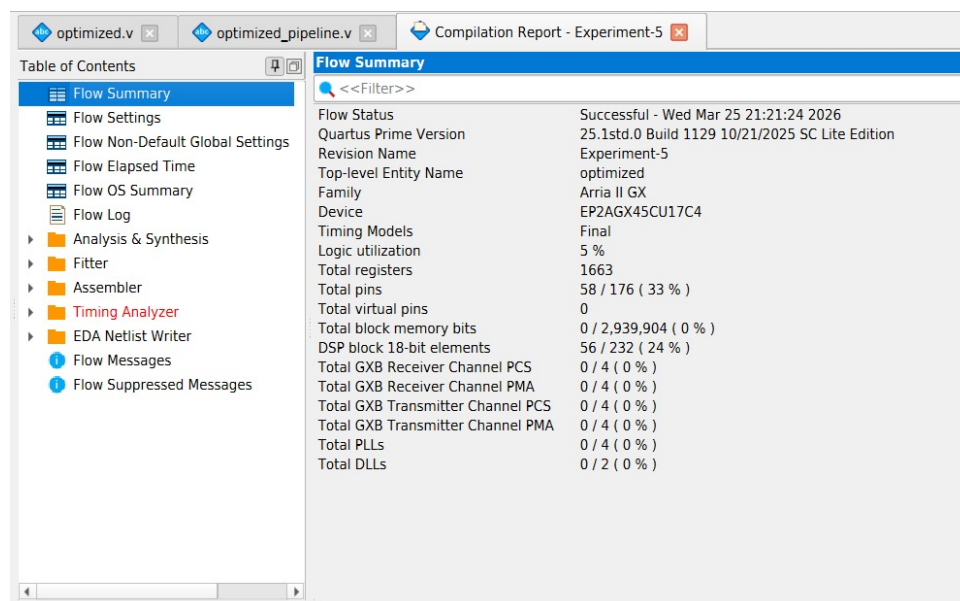
The synthesized results obtained from Quartus are shown below.

7.1 Non-Pipelined Designs



Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Mar 25 21:10:39 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	Experiment-5
Top-level Entity Name	direct
Family	Arria II GX
Device	EP2AGX45CU17C4
Timing Models	Final
Logic utilization	5 %
Total registers	1624
Total pins	58 / 176 (33 %)
Total virtual pins	0
Total block memory bits	0 / 2,939,904 (0 %)
DSP block 18-bit elements	96 / 232 (41 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 2 (0 %)

Figure 2: Direct Form (Non-Pipeline)



Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Mar 25 21:21:24 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	Experiment-5
Top-level Entity Name	optimized
Family	Arria II GX
Device	EP2AGX45CU17C4
Timing Models	Final
Logic utilization	5 %
Total registers	1663
Total pins	58 / 176 (33 %)
Total virtual pins	0
Total block memory bits	0 / 2,939,904 (0 %)
DSP block 18-bit elements	56 / 232 (24 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 2 (0 %)

Figure 3: Optimized Form (Non-Pipeline)

The screenshot shows the 'Flow Summary' window in Quartus Prime. The left sidebar contains a 'Table of Contents' with the following items: Flow Summary (selected), Flow Settings, Flow Non-Default Global Settings, Flow Elapsed Time, Flow OS Summary, Flow Log, Analysis & Synthesis, Fitter, Assembler, Timing Analyzer, EDA Netlist Writer, Flow Messages, and Flow Suppressed Messages. The main area displays the following data:

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Mar 25 21:34:19 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	Experiment-5
Top-level Entity Name	fir_genvar
Family	Arria II GX
Device	EP2AGX45CU17C4
Timing Models	Final
Logic utilization	5 %
Total registers	1624
Total pins	58 / 176 (33 %)
Total virtual pins	0
Total block memory bits	0 / 2,939,904 (0 %)
DSP block 18-bit elements	100 / 232 (43 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 2 (0 %)

Figure 4: Genvar Form (Non-Pipeline)

7.2 Pipelined Designs

The screenshot shows the 'Flow Summary' window in Quartus Prime for a pipelined design. The left sidebar is identical to Figure 4. The main area displays the following data:

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Mar 25 21:15:37 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	Experiment-5
Top-level Entity Name	direct_pipeline
Family	Arria II GX
Device	EP2AGX45CU17C4
Timing Models	Final
Logic utilization	29 %
Total registers	9190
Total pins	58 / 176 (33 %)
Total virtual pins	0
Total block memory bits	0 / 2,939,904 (0 %)
DSP block 18-bit elements	0 / 232 (0 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 2 (0 %)

Figure 5: Direct Form Pipeline

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Mar 25 21:25:40 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	Experiment-5
Top-level Entity Name	optimized_pipeline
Family	Arria II GX
Device	EP2AGX45CU17C4
Timing Models	Final
Logic utilization	18 %
Total registers	7078
Total pins	58 / 176 (33 %)
Total virtual pins	0
Total block memory bits	10,200 / 2,939,904 (< 1 %)
DSP block 18-bit elements	100 / 232 (43 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 2 (0 %)

Figure 6: Optimized Pipeline

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Mar 25 22:23:56 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	Experiment-5
Top-level Entity Name	genvar_pipeline
Family	Arria II GX
Device	EP2AGX45CU17C4
Timing Models	Final
Logic utilization	29 %
Total registers	9204
Total pins	58 / 176 (33 %)
Total virtual pins	0
Total block memory bits	0 / 2,939,904 (0 %)
DSP block 18-bit elements	0 / 232 (0 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 2 (0 %)

Figure 7: Genvar Pipeline

8 Analysis of Results and Performance

8.1 Resource Utilization Analysis (Quartus Results)

The FIR filter designs were synthesized using Intel Quartus Prime Lite, and the hardware resource utilization was obtained.

The results show that pipelined designs consume more hardware resources compared to non-pipelined designs. This increase is mainly due to the insertion of pipeline registers at different stages of the design.

- **Registers:** The number of registers increases significantly in pipelined designs due to storage elements between stages.

- **Logic Elements (ALMs):** Logic usage also increases because of additional control and pipeline structures.
- **DSP Blocks:** The number of DSP blocks remains approximately the same, since the number of multiplications does not change significantly.

8.2 Pipeline Efficiency

Pipeline architecture improves the performance of the FIR filter by increasing throughput.

- Once the pipeline is filled, the system produces one output per clock cycle.
- This results in high data processing speed.
- The overall efficiency improves for continuous data streams.

However, this improvement comes at the cost of increased hardware usage.

8.3 Working of Pipeline Architecture

In a pipelined FIR filter, the computation is divided into multiple stages:

1. Input samples are stored in a shift register (delay line).
2. Each stage performs multiplication with filter coefficients.
3. Partial results are passed through pipeline registers.
4. Final output is obtained after multiple clock cycles.

Each pipeline stage operates simultaneously on different data samples, similar to an assembly line, which increases parallelism.

8.4 Latency and Throughput

- **Latency:** The time taken for an input sample to produce an output. In pipelined FIR filters, latency is equal to the number of pipeline stages (approximately equal to the number of taps).
- **Throughput:** The rate at which outputs are produced. In pipelined systems, throughput is high because one output is generated every clock cycle after pipeline filling.

Thus, pipelining increases throughput but also increases latency.

8.5 Quartus Design Flow

The following steps were followed in Quartus Prime Lite:

1. Verilog design files were created for FIR filter architectures.
2. The project was compiled using Quartus to perform synthesis and fitting.
3. Resource utilization was obtained from the compilation report.
4. The design was analyzed for logic usage, registers, and DSP blocks.
5. Simulation was performed using ModelSim to verify functionality.

8.6 Overall Observation

- Pipeline architecture improves speed and performance.
- It increases hardware complexity and resource usage.
- There is a trade-off between performance and hardware cost.
- The obtained results from Quartus and MATLAB confirm correct operation of the FIR filter.

9 Observations

- Pipelined designs use more registers due to additional pipeline stages.
- Logic utilization increases in pipelined implementations.
- Pipeline architecture improves system performance by reducing critical path delay.
- Output waveforms obtained from MATLAB confirm correct filter operation.

10 Conclusion

The pipelined FIR filter was successfully implemented using Verilog.

Resource utilization was analyzed using Quartus, and output results were verified using MATLAB. The pipeline architecture improved performance at the cost of increased hardware usage.